

A13. Pràctica: Criptografia amb OpenSSL

0226. Seguretat informàtica

GRUP: SMX2C

RA3. Implanta mecanismes de seguretat activa aplicant tècniques i procediments per evitar accions malicioses, i garanteix la confidencialitat i integritat de la informació mitjançant tècniques criptogràfiques.

NOM I COGNOMS: Posa el teu nom i cognoms.

DATA: Escriu la data.

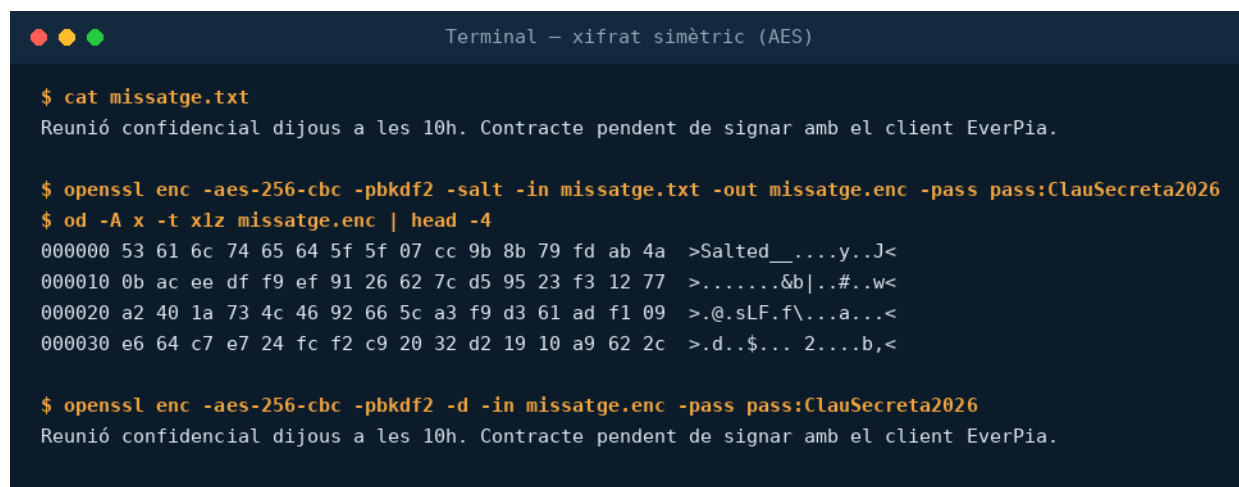
Enunciat de l'activitat:

Aquesta activitat es cenyeix a l'apartat NF3-AA3 (Criptografia) del RA3: xifrat simètric, funcions hash i signatura digital, practicats amb OpenSSL. Totes les captures d'aquesta guia s'han fet expressament, executant realment les ordres indicades: reproduïu els mateixos passos al teu equip (Linux, o Windows amb OpenSSL instal·lat) i compara els resultats.

Part 1 · Xifrat simètric (AES-256)

Pas 1. Crea un fitxer de text amb un missatge confidencial qualsevol (per exemple, missatge.txt) i xifra'l amb AES-256 mitjançant OpenSSL:

```
$ openssl enc -aes-256-cbc -pbkdf2 -salt \
-in missatge.txt -out missatge.enc \
-pass pass:LaTevaContrasenyaSecreta
```



```
Terminal - xifrat simètric (AES)

$ cat missatge.txt
Reunió confidencial dijous a les 10h. Contracte pendent de signar amb el client EverPia.

$ openssl enc -aes-256-cbc -pbkdf2 -salt -in missatge.txt -out missatge.enc -pass pass:ClauSecreta2026
$ od -A x -t x1z missatge.enc | head -4
000000 53 61 6c 74 65 64 5f 5f 07 cc 9b 8b 79 fd ab 4a >Salted____.y..J<
000010 0b ac ee df f9 ef 91 26 62 7c d5 95 23 f3 12 77 >.....&b|..#..w<
000020 a2 40 1a 73 4c 46 92 66 5c a3 f9 d3 61 ad f1 09 >.@.sLF.f\...a...<
000030 e6 64 c7 e7 24 fc f2 c9 20 32 d2 19 10 a9 62 2c >.d..$.... 2....b,<

$ openssl enc -aes-256-cbc -pbkdf2 -d -in missatge.enc -pass pass:ClauSecreta2026
Reunió confidencial dijous a les 10h. Contracte pendent de signar amb el client EverPia.
```

Captura real: xifrat AES-256 d'un missatge de text, visualització del fitxer xifrat en hexadecimal (il·legible), i desxifrat correcte amb la mateixa contrasenya.

Fes-ho amb el teu propi missatge [CAPTURA] i respon: a) Si obres el fitxer .enc amb un editor de text normal, què hi veus? b) Què passaria si intentes desxifrar-lo amb una contrasenya diferent de l'original? Prova-ho i explica el resultat.

Part 2 · Generació d'un parell de claus RSA

Pas 2. Genera un parell de claus RSA de 3072 bits (clau privada i, a partir d'ella, la clau pública corresponent):

```
$ openssl genrsa -out alice_priv.pem 3072
$ openssl rsa -in alice_priv.pem -pubout -out alice_pub.pem
```

```
Terminal – generació de claus RSA

$ openssl genrsa -out alice_priv.pem 3072

$ openssl rsa -in alice_priv.pem -pubout -out alice_pub.pem
writing RSA key

$ cat alice_pub.pem
-----BEGIN PUBLIC KEY-----
MIIBojANBgkqhkiG9w0BAQEFAAOCAY8AMIIBigKCAYEA0ZIW0uxllukYkVMDt7+l
u6IGQ5w0iIatm2q2SZsPel+codobMvRhY71HK7H/t25yssHMsCKwGdcr3tK1V43i
Ae05wwNsJ1m7BBuZYb7CXiDadH3DTqplQjPcOP6LrwGnZme93NcGC4g8bfIwTW8a
7wGsJuIloB7RtUltQNYCUIJNbh/69qcLBLeJtUD0ivS9VmnnN2jzJ5VgiYtUfLzX
4d0t05kllv2kliCGB1cdnscb3SKkDzIbHTDHt74MFRvj00tjt9qveBXTXS34+0QW
JHRkBDRyu6hrqwSdPC/v0dpGw5atIFyRKUnvfyB0eNeQb3veeLQrYEimvMDdAeqz
aXoFvnLwqLoYfHhz/NewMES80P+H1/M3Bq3G/JHmzx9GZFLE0+fPDKcXDq6dz0mK
MNJyXSyM110fMmCai/4fR88llgw4YQejh98mQsdhqtWmREHRrvnUdMkbh0WslgIA
RhiL40mpTdU60EwAJPQPZXDaEznW1jK9MsgV5jEQ/ZldAgMBAAE=
-----END PUBLIC KEY-----
```

Captura real: generació d'una clau privada RSA de 3072 bits i extracció de la clau pública corresponent (el contingut del fitxer .pem es mostra codificat en Base64).

Genera el teu propi parell de claus [CAPTURA] i respon: a) Quina de les dues claus (alice_priv.pem o alice_pub.pem) hauries de mantenir en secret? b) Quina compartiries amb altres persones perquè et puguin enviar missatges xifrats o verificar les teves signatures?

Part 3 · Signatura digital: signar i verificar

Pas 3. Signa digitalment un document (el hash SHA-256, amb la teva clau privada) i verifica-la amb la clau pública:

```
$ echo "Contracte EverPia v3 - import 12500 EUR" > contracte.txt
$ sha256sum contracte.txt
$ openssl dgst -sha256 -sign alice_priv.pem \
  -out contracte.sig contracte.txt
$ openssl dgst -sha256 -verify alice_pub.pem \
  -signature contracte.sig contracte.txt
```

Pas 4. Ara simula que algú ha modificat el contracte DESPRÉS de signar-lo (sense tornar-lo a signar), i intenta verificar-lo de nou:

```
Terminal - signatura digital

$ echo "Contracte EverPia v3 - import 12500 EUR" > contracte.txt

$ sha256sum contracte.txt
e9cf3c2d8bc046e08024392e4baed23040302f0539570169b00f8fc1017ecf14  contracte.txt

$ openssl dgst -sha256 -sign alice_priv.pem -out contracte.sig contracte.txt

$ openssl dgst -sha256 -verify alice_pub.pem -signature contracte.sig contracte.txt
Verified OK

# Simulem que algú modifica el document DESPRES de signar-lo:
$ openssl dgst -sha256 -verify alice_pub.pem -signature contracte.sig contracte.txt
4047CA705D7F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:430:
4047CA705D7F0000:error:1C880004:Provider routines:rsa_verify:RSA lib:../providers/implementations/signature/rsa_sig.c:774:
Verification failure
```

Captura real: signatura i verificació correcta ("Verified OK") d'un contracte de text; després, en modificar l'import del contracte SENSE tornar-lo a signar, la verificació falla ("Verification failure").

Reprodueix aquest mateix experiment amb un fitxer propi [CAPTURA] i respon: a) Per què la verificació falla després de modificar el document, si la clau pública i la signatura no han canviat? b) Relaciona aquest resultat amb el concepte d'INTEGRITAT que vas estudiar. c) Si en lloc de modificar el contingut, algú intentés verificar la signatura amb la clau pública d'una ALTRA persona, què creus que passaria? Prova-ho si tens temps.

Reflexió final

Redacta un breu informe (mitja pàgina) explicant, amb les teves paraules, per què la signatura digital garanteix ALHORA autenticitat (l'emissor és qui diu ser), integritat (el document no s'ha alterat) i no-repudi (l'emissor no pot negar haver-lo signat), relacionant cada garantia amb un pas concret del procediment que has practicat (generació de claus, signatura, verificació).

Fonts: Repositori "Materials" (GitHub, SMX-0226SI), NF3/AA3-Criptografia.md (CC-BY-SA-4.0); OpenSSL (openssl.org); Practical Networking, "Cryptography".